SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5103

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

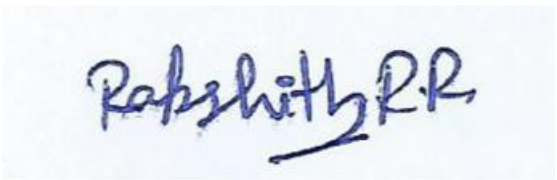
Scenario-Based : 20%

QUESTIONS: Total Marks: 50 Annexure A

Code of Conduct:

*I **RAKSHITH R.R (192324156)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



Annexure A
Scenario-based

1. Secure Data Transmission using Block Cipher
A company is transmitting repeated financial data blocks over a network. Initially, they used ECB mode and noticed patterns in ciphertext.

A. Explain why this issue occurs.
B. Suggest a more secure block cipher mode and justify your choice based on the scenario.

2. Choosing an Encryption Algorithm for Banking System
A banking application needs high security and moderate performance for encrypting customer transactions. The available options are DES, 3-DES, and Blowfish.

A. Identify the most suitable algorithm for this scenario.
B. Justify your choice based on security strength and efficiency.

3. Key Exchange in an Unsecured Network
Two users want to establish a secure communication channel over a public network using Diffie-Hellman key exchange. However, the network is vulnerable to attacks.

A. Explain how the key exchange works in this scenario.
B. Identify a possible security threat and suggest a solution to mitigate it.

4. Public Key Cryptography for Email Security
An organization plans to implement RSA for secure email communication.

A. Describe how RSA can be used for encryption and decryption in this scenario.
B. Explain how key distribution and management play a role in maintaining security.

5. Cryptography for IoT Devices
A smart home system uses low-power IoT devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC).

A. Analyze the constraints of IoT devices.
B. Recommend a suitable cryptographic approach and justify your answer.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well reasoned, logical decisions with	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided

		strong rationale			
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganiz ed and difficult to understand

Question 1 – Secure Data Transmission using Block Cipher

Scenario: A company transmits repeated financial data blocks over a network. It first used ECB mode and noticed patterns in the ciphertext. (A) Explain why this issue occurs. (B) Suggest a more secure block cipher mode and justify the choice for this scenario.

1.1 Understanding of the Scenario

A block cipher such as AES encrypts exactly one fixed-size block (128 bits = 16 bytes) at a time. Real messages are longer than one block, so a mode of operation decides how the blocks are processed. The company chose ECB (Electronic Codebook), the simplest mode, and its network traffic is financial data, which is highly structured and repetitive (the same account numbers, the same transaction headers, and often the same amounts). The pattern the company noticed is therefore not a fault in AES itself but a property of the mode.

Given in the scenario	What it means for the analysis
Repeated financial data blocks	The plaintext contains identical 16-byte blocks, so any deterministic mode will expose the repetition.
ECB mode used	Each block is encrypted independently with the same key; no IV, nonce or chaining value is mixed in.
Patterns noticed in ciphertext	An eavesdropper can see structure without knowing the key – a confidentiality failure.
Data is financial, sent over a network	Needs confidentiality, tamper detection and good throughput; the attacker can sit on the network.

1.2 Identification of Key Issues

#	Issue	Impact on the company
1	**Deterministic encryption**	Equal plaintext blocks always give equal ciphertext blocks under one key.
2	**Pattern leakage**	An observer learns which transactions repeat (for example the same payment sent several times) and can build a codebook of block → meaning.
3	**No diffusion between blocks**	Changing one block affects only its own ciphertext block, so structure survives (shown by the image test in Figure 2).
4	**Block substitution / replay**	Blocks can be swapped, repeated or removed and still decrypt without any error (demonstrated in Table 2).
5	**No integrity or authentication**	ECB alone cannot detect tampering – unacceptable for financial records.
6	**Wrong mode, not wrong cipher**	Switching to a bigger key or a different cipher will not fix it; the mode must change.

1.3 Application of Concepts – Part A: Why the Issue Occurs

Step 1: Write the ECB rule. ECB splits the message into blocks P1, P2, … and computes Ci = EK(Pi) for each block, using the same key K every time. Decryption is Pi = DK(Ci). No other input changes from block to block.

Step 2: Recognise what EK is. For a fixed key, EK is a fixed one-to-one function (a permutation) on 128-bit values. Giving it the same input twice must give the same output.

Step 3: Apply it to the company's data. Four 16-byte records were encrypted with AES-128 in ECB mode. Records 1, 2 and 4 are identical:

Block	Plaintext (16 bytes)	AES-128 ECB ciphertext (hex, first 8 bytes)	Observation
P1 / C1	PAY ACC1001 5000	4c1d1307b03c783e…	**same**
P2 / C2	PAY ACC1001 5000	4c1d1307b03c783e…	**same as C1**
P3 / C3	PAY ACC2002 9000	cac7222cd4ab7991…	**different**
P4 / C4	PAY ACC1001 5000	4c1d1307b03c783e…	**same as C1**

Table 1: Real AES-128-ECB output – three of the four ciphertext blocks are identical

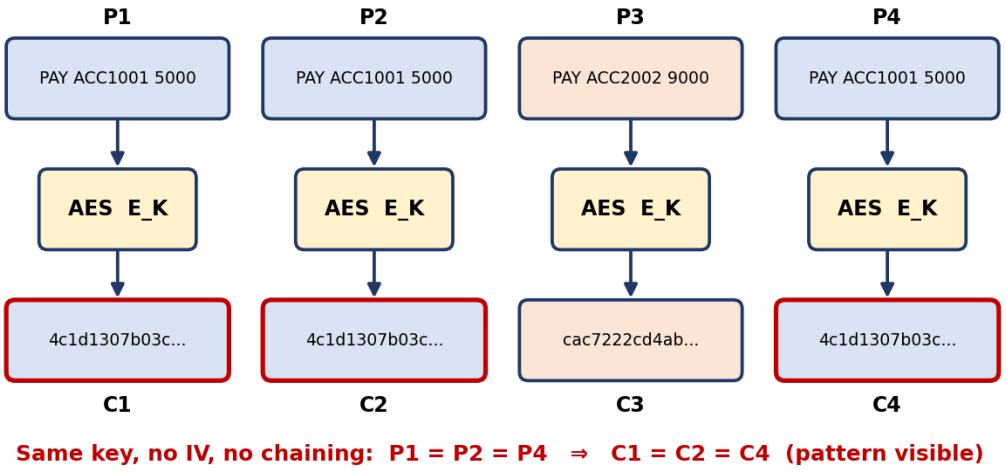


Figure 1: ECB mode – every block is encrypted independently, so repeated plaintext shows up as repeated ciphertext

Step 4: See the pattern leak visually. A greyscale picture was encrypted byte-for-byte with the same AES key in three modes (Figure 2). In ECB the outline of the circle, rectangle and triangle remains clearly visible because large uniform areas produce the same ciphertext block over and over. CBC and CTR show only noise.

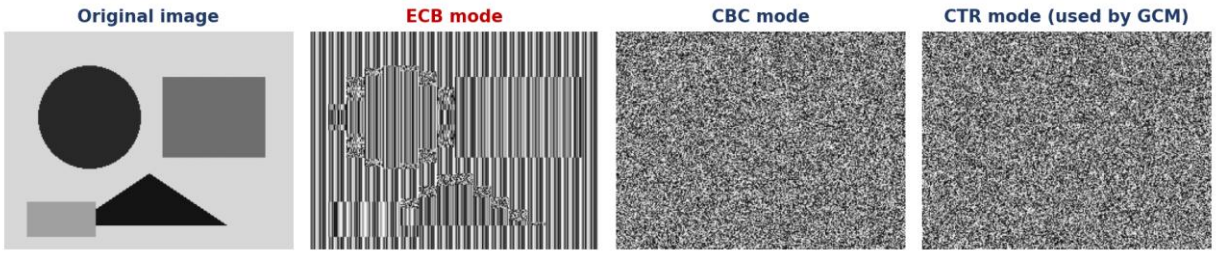


Figure 2: The same image encrypted in ECB, CBC and CTR modes

Step 5: Show the second danger – block manipulation. An attacker on the network swaps ciphertext block C1 with C3. The receiver decrypts without any error and obtains the records in a changed order:

Position	Original records (sent)	Records after attacker swaps C1 ↔ C3
Block 1	PAY ACC1001 5000	PAY ACC2002 9000
Block 2	PAY ACC1001 5000	PAY ACC1001 5000
Block 3	PAY ACC2002 9000	PAY ACC1001 5000
Block 4	PAY ACC1001 5000	PAY ACC1001 5000

Table 2: ECB accepts reordered blocks silently (no integrity protection)

Answer to Part A

The issue occurs because ECB is deterministic. Each block is encrypted independently with the same key and with no IV, counter or chaining, so identical plaintext blocks always produce identical ciphertext blocks. Repetitive financial data therefore leaves visible patterns, lets an attacker recognise repeated transactions and build a block dictionary, and allows blocks to be swapped or replayed undetected. The weakness lies in the mode of operation, not in AES.

1.4 Decision Making – Part B: A More Secure Mode

Options considered

Mode	Hides repeated blocks?	Needs IV / nonce	Integrity check	Parallel encryption	Verdict for this scenario
ECB	**No**	No	No	Yes	**Rejected**
CBC	Yes	Random IV	No (needs HMAC)	Decrypt only	Acceptable, extra work
CTR	Yes	Unique nonce	No (needs HMAC)	Yes	Good, still no tag
GCM	**Yes**	Unique nonce	**Yes (128-bit tag)**	**Yes**	**Selected**

Table 3: Comparison of block cipher modes

How the runner-up (CBC) works

In CBC each plaintext block is XORed with the previous ciphertext block before encryption: Ci = EK(Pi ⊕ Ci−1), with C0 = IV (a random value). Because Ci−1 differs for every block, even identical plaintext blocks encrypt differently. The drawback is that CBC gives no integrity, needs padding, encrypts strictly one block after another, and has a history of padding-oracle attacks when combined with poor error handling.

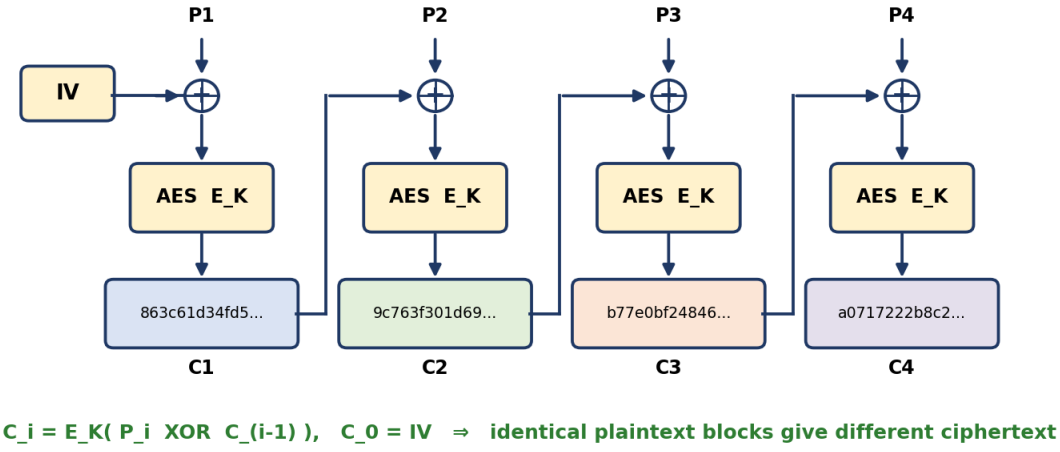


Figure 3: CBC mode – ciphertext chaining removes the repeated-block pattern

Recommended mode: AES in Galois/Counter Mode (AES-GCM)

GCM turns AES into a counter-mode stream (each 16-byte block is XORed with EK(nonce ‖ counter), and the counter changes every block) and adds GHASH, which produces a 128-bit authentication tag over the header and the whole ciphertext (Figure 4). It is an authenticated-encryption (AEAD) mode standardised in NIST SP 800-38D and is used in TLS 1.2/1.3 and IPsec.

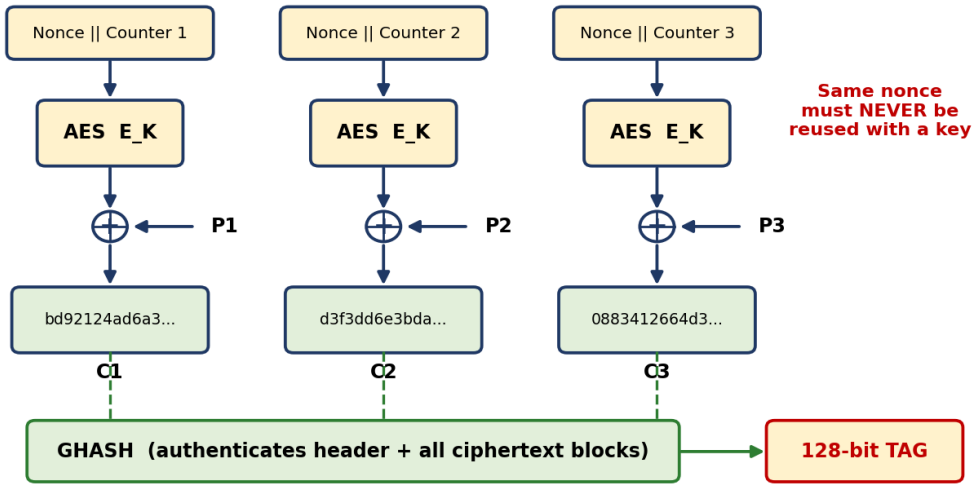


Figure 4: AES-GCM – counter-mode encryption plus an authentication tag

Evidence: the same four records re-encrypted

Block	Plaintext	ECB	CBC (random IV)	GCM (unique nonce)
1	PAY ACC1001 5000	4c1d1307b03c783e…	863c61d34fd51937…	bd92124ad6a3ed37…
2	PAY ACC1001 5000	4c1d1307b03c783e…	9c763f301d69f06e…	d3f3dd6e3bdabb4b…
3	PAY ACC2002 9000	cac7222cd4ab7991…	b77e0bf24846d6e7…	0883412664d3ca1f…
4	PAY ACC1001 5000	4c1d1307b03c783e…	a0717222b8c26df1…	22062e4270a18b86…
Repeats		**2 repeated blocks**	**0 repeats**	**0 repeats + tag**

Table 4: Real ciphertext in three modes (GCM tag = f65e5a318e81eacd…)

Tamper test: one bit of the GCM ciphertext was flipped. The receiver's decryption result was: **REJECTED (InvalidTag)**. In ECB the swapped blocks were accepted (Table 2), so GCM closes the integrity gap as well.

Justification based on the scenario

Scenario need	Why AES-GCM satisfies it
Repeated blocks must not be visible	The keystream block changes with the counter, so equal plaintext blocks give different ciphertext (0 repeats in Table 4).
Financial data must not be altered	The 128-bit tag detects any modification, swap or truncation; tampered messages are rejected before use.
Network throughput matters	Blocks can be processed in parallel, no padding is needed, and AES-NI / carry-less multiply hardware makes it very fast.
Header fields must stay readable but protected	Associated data (for example transaction ID) is authenticated without being encrypted.
Standards and audit requirements	NIST-approved and mandatory-to-support in TLS 1.3, which simplifies compliance.

Safe-use rules (implementation steps)

Step 1: Key: use a random 128- or 256-bit AES key from a key-management system; rotate it regularly.

Step 2: Nonce: use a unique 96-bit nonce for every message under the same key (counter or random); never repeat it, because nonce reuse destroys GCM security.

Step 3: Send: transmit nonce ‖ ciphertext ‖ tag, and put the transaction header in the associated data.

Step 4: Receive: verify the tag first; if it fails, discard the message and raise an alert.

Step 5: Fallback: if GCM is unavailable, use AES-CBC with a random IV plus HMAC-SHA-256 (encrypt-then-MAC); never use ECB.

Answer to Part B

Replace ECB with AES-GCM (authenticated encryption). It randomises every block through a unique nonce and counter so repeated financial records no longer match, and it adds an authentication tag that detects tampering, with parallel, padding-free, hardware-accelerated performance. AES-CBC with a random IV plus HMAC is the acceptable alternative; ECB should not be used for this data.

Question 5 – Cryptography for IoT Devices

Scenario: A smart home system uses low-power IoT devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC). (A) Analyze the constraints of IoT devices. (B) Recommend a suitable cryptographic approach and justify the answer.

2.1 Understanding of the Scenario

A smart home contains many small battery-powered devices (door locks, sensors, bulbs, cameras) that talk to a home hub and, through it, to a cloud service or phone app. Public-key cryptography is needed for two jobs: establishing a shared session key without a prior secret (key exchange) and proving identity (digital signatures or certificates). Bulk data is then protected with a fast symmetric cipher. The designer's question is which public-key family, RSA or ECC, costs the devices less while giving the same protection.

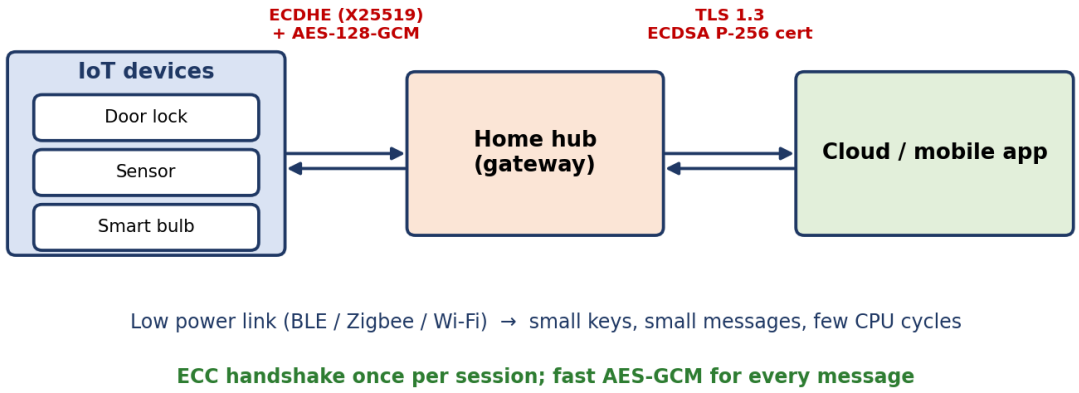


Figure 5: Smart-home architecture and the proposed cryptography on each link

2.2 Identification of Key Issues

#	Issue	Why it matters for the RSA / ECC decision
1	**Very small processing and memory budget**	Big-number operations on 2048-bit values are heavy for a small microcontroller.
2	**Battery and energy limits**	Every extra CPU millisecond and radio byte shortens battery life.
3	**Low bandwidth and small packets**	Keys, certificates and signatures must fit small frames (for example 127-byte IEEE 802.15.4 frames).

4	**Large scale and long life**	Thousands of devices need cheap key generation, provisioning and years of safe operation.
5	**Physical exposure and weak randomness**	Devices can be stolen or probed (side-channel attacks), and low-cost hardware may give poor random numbers.
6	**Threats: eavesdropping, spoofing, replay, firmware tampering**	Need confidentiality, authentication, integrity and freshness at low cost.

2.3 Application of Concepts – Part A: Constraints of IoT Devices

Step 1: List the device limits. The IETF classification of constrained devices (RFC 7228) places typical Class 1 devices at roughly 10 KiB of RAM and 100 KiB of flash, and Class 0 devices well below that. Processors are often 8- or 32-bit microcontrollers running at tens of MHz.

Step 2: Translate each limit into a requirement on the cryptography. See Table 5.

Constraint	Typical value	Requirement on the cryptography
CPU	8/32-bit MCU, 10–100 MHz, often no big-number hardware	Cheap private-key operations (sign, key agreement) and fast key generation
Memory	Class 0/1: about 10 KiB RAM, 100 KiB flash	Small keys and small code; no large number buffers
Energy	Coin cell or battery, years of life	Few CPU cycles and few radio bytes per handshake
Network	Zigbee 250 kbps; 127-byte 802.15.4 frames; BLE and LoRa are also low rate	Small certificates, public keys and signatures; short handshake
Lifetime / scale	5–10+ years, thousands of nodes	Strong security per bit, crypto-agility, simple provisioning
Physical / RNG	Unattended, cheap hardware	Side-channel-resistant, safe with weak randomness, secure element if possible

Table 5: Constraints of IoT devices and what they demand from cryptography

Step 3: Compare RSA and ECC at equal security. To reach 128-bit security NIST SP 800-57 requires a 3072-bit RSA key but only a 256-bit ECC key (Figure 6). At every level ECC keys are 6 to 30 times smaller.

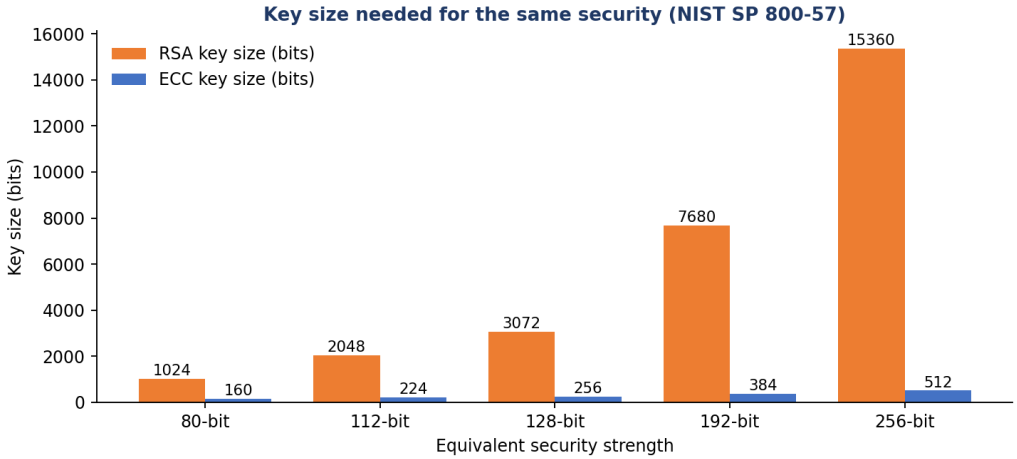


Figure 6: Key size needed for equal security (NIST SP 800-57)

Step 4: Measure the cost. RSA-2048 and ECC P-256 (both about 112–128-bit security) were timed with the Python cryptography library, and sizes were taken from the generated keys and signatures (Table 6, Figure 7).

Measurement	RSA-2048	ECC P-256	Result
Key generation (ms)	61.0	0.020	**ECC ≈ 3,044× faster**
Sign with private key (ms)	0.395	0.033	**ECC ≈ 12× faster**
Verify with public key (ms)	0.026	0.092	**RSA faster**
Public key size (bytes, DER)	294	91	**ECC ≈ 3.2× smaller**
Signature size (bytes)	256	72	**ECC ≈ 3.6× smaller**

Table 6: Measured comparison (test PC; timings show relative cost, absolute times on a microcontroller are far larger)

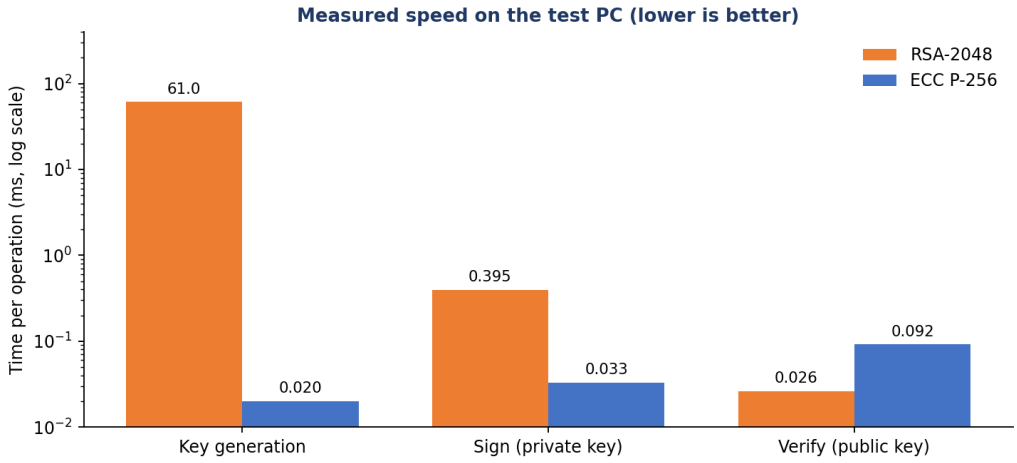


Figure 7: Measured time per operation (logarithmic scale)

Step 5: Interpret honestly. RSA verification is quick because its public exponent is small, so RSA is not slow at everything. But a smart-home device also has to generate keys, sign and agree keys, and there RSA

is 12 to thousands of times slower. A 256-byte RSA signature alone would not fit in one 127-byte radio frame, whereas a 72-byte ECDSA signature does.

Answer to Part A

IoT devices are constrained in CPU, memory, energy, network bandwidth and physical security, and they are deployed in large numbers for many years. Public-key cryptography must therefore use small keys and messages, cheap private-key operations, fast key generation and strong security per bit, while also coping with weak random-number sources and exposed hardware.

2.4 Decision Making – Part B: Recommended Approach

Scored decision matrix

Criterion	Weight %	ECC (1–5)	RSA (1–5)
Computation and energy	25	5	2
Key and message size	20	5	2
Security per key bit	20	5	2
Handshake bandwidth	15	5	2
Maturity and compatibility	10	4	5
Implementation risk	10	3	4
Weighted score	**100**	**4.70**	**2.50**

Table 7: Weighted decision matrix (scores assigned from the evidence in Section 2.3)

RSA scores higher only on maturity and implementation simplicity; ECC wins every criterion that reflects the real constraints of the devices.

Recommendation

Use ECC for the public-key part (ephemeral X25519 or ECDH P-256 key agreement, with ECDSA P-256 or Ed25519 signatures for authentication) and use an AES-128-GCM session (or ChaCha20-Poly1305 on devices without AES hardware) for the data. In practice this is TLS 1.3 or DTLS 1.3 with an ECDHE-ECDSA suite, or an equivalent lightweight protocol.

How a session works (step by step)

Step 1: Provisioning: each device receives an ECC key pair and certificate at manufacture, ideally stored in a secure element.

Step 2: Handshake: device and hub each create a temporary X25519 key pair and exchange the 32-byte public keys; each side signs the handshake with its long-term ECC key so the other can authenticate it.

Step 3: Shared secret: both compute the same shared value (verified equal in the test) and derive a session key with HKDF-SHA-256 (test key: 35bdaf61a250128e…).

Step 4: Protected data: each message is sent with AES-GCM and a fresh nonce. The test message "temperature=24.5C;door=locked" of 29 bytes became 45 bytes (only 16 bytes of tag overhead) and decrypted correctly.

Step 5: Renewal: session keys are replaced regularly; the ephemeral keys give forward secrecy, so a stolen device cannot decrypt past traffic.

Justification

Reason	Evidence from this analysis
Smaller keys and messages	Public key 91 B vs 294 B and signature 72 B vs 256 B (Table 6); 256-bit ECC equals 3072-bit RSA (Figure 6). The ECC signature fits in one radio frame.
Lower CPU and energy cost	Signing is about 12× and key generation thousands of times faster than RSA-2048, so less battery is used per handshake.
Short handshake	Fewer bytes on a slow link, shorter wake time for sleeping devices.
Future headroom	Security can be raised (P-384) at far lower cost than moving RSA from 2048 to 7680 bits.
Industry practice	TLS 1.3, Matter and most modern IoT stacks use ECC.

Risks and mitigations

Risk	Mitigation
Bad random numbers break ECDSA	Use a hardware RNG and deterministic signatures (Ed25519 or RFC 6979).
Side-channel attacks on stolen devices	Constant-time libraries and a secure element or TPM-class chip.
Weak or non-standard curves	Use only standard curves: P-256 or Curve25519.
Quantum computers (Shor's algorithm breaks both RSA and ECC)	Design for crypto-agility so post-quantum algorithms can be added through firmware updates.
Firmware tampering	Signed firmware, secure boot. RSA is still acceptable here because devices only verify, which is cheap.

Answer to Part B

ECC is the more suitable choice. For the same security it uses much smaller keys (256-bit ECC versus 3072-bit RSA), signatures and handshakes, and much cheaper private-key operations and key generation, which saves processing time, memory, bandwidth and battery. The recommended design is an ECDHE (X25519 or P-256) key exchange authenticated with ECDSA/Ed25519, followed by AES-128-GCM for data, with a hardware random source, constant-time code and crypto-agility to handle side-channel and quantum risks.

Overall Conclusion

Question 1 showed that ECB leaks patterns and permits undetected block manipulation because every block is encrypted identically and independently; AES-GCM is the better choice because it hides repeats and authenticates the data. Question 5 showed that, although RSA verification is cheap, ECC is the better public-key family for low-power smart-home devices because of its small keys, small signatures and low computation and energy cost. In both cases the secure design combines a sound public-key or mode choice with correct operational rules (unique nonces, good randomness, key management).

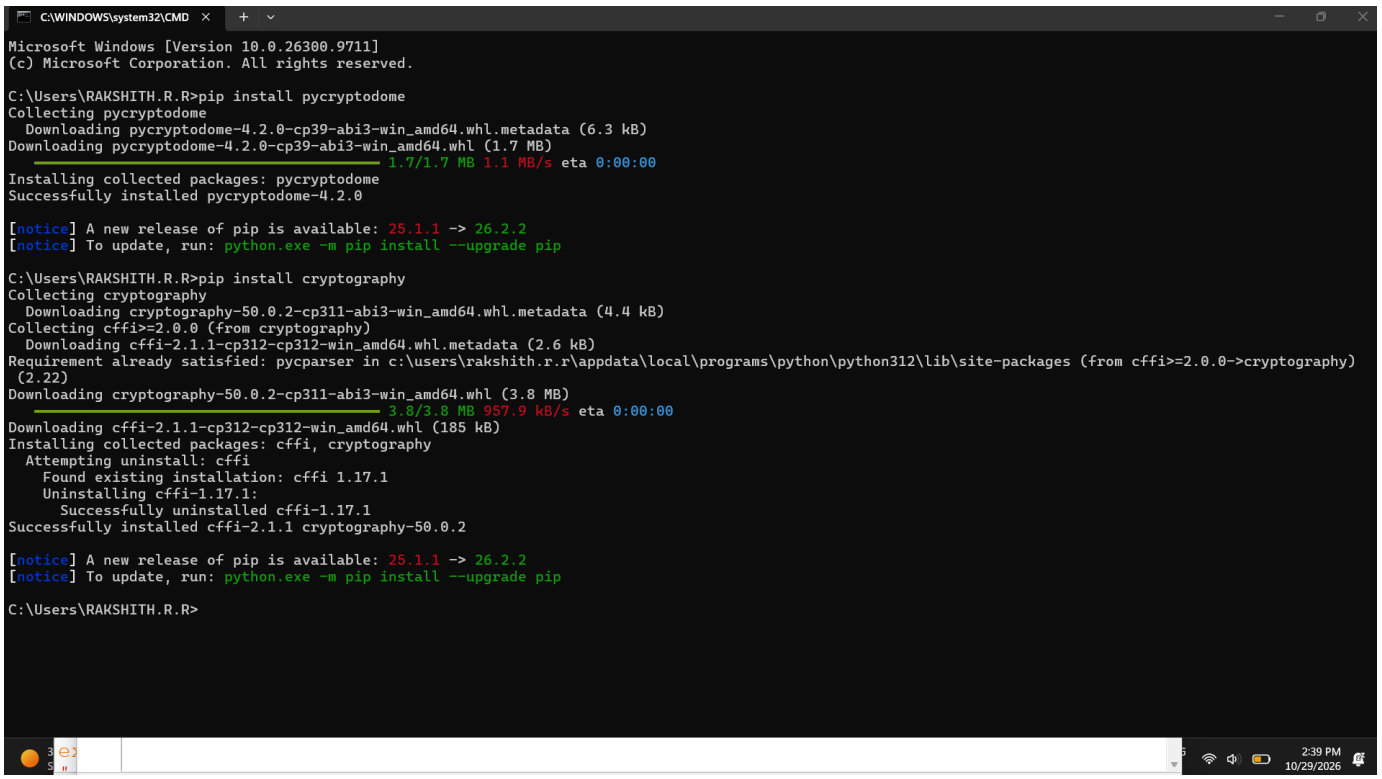
Appendix – Python Program Used for the Experiments

```
import os, time, json
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.asymmetric import rsa, ec, padding, x25519, ed25519
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.kdf.hkdf import HKDF

res = {}
key = bytes.fromhex("000102030405060708090a0b0c0d0e0f")
iv  = bytes.fromhex("a0a1a2a3a4a5a6a7a8a9aaabacadaeaf")
# financial records: 16-byte blocks
blk = lambda s: s.encode().ljust(16)[:16]
blocks = [blk("PAY ACC1001 5000"), blk("PAY ACC1001 5000"), blk("PAY ACC2002 9000"),
blk("PAY ACC1001 5000")]
pt = b"".join(blocks)
def enc(mode):
    c = Cipher(algorithms.AES(key), mode).encryptor(); return c.update(pt) + c.finalize()
ecb = enc(modes.ECB()); cbc = enc(modes.CBC(iv)); ctr = enc(modes.CTR(iv))
split = lambda b: [b[i:i+16].hex() for i in range(0, len(b), 16)]
res["blocks"] = [b.decode().strip() for b in blocks]
res["ecb"] = split(ecb); res["cbc"] = split(cbc); res["ctr"] = split(ctr)
nonce = bytes.fromhex("001122334455667788 99aabb")
g = AESGCM(key).encrypt(nonce, pt, b"txn-header")
res["gcm_ct"] = split(g[:-16]); res["gcm_tag"] = g[-16:].hex()
# tamper test for ECB (swap block 3 and 1 -> still decrypts) vs GCM (rejects)
sw = ecb[32:48] + ecb[16:32] + ecb[0:16] + ecb[48:]
d = Cipher(algorithms.AES(key), modes.ECB()).decryptor(); swp = d.update(sw) + d.finalize()
res["ecb_swapped_plain"] = [swp[i:i+16].decode().strip() for i in range(0, 64, 16)]
bad = bytearray(g); bad[5] ^= 1
try: AESGCM(key).decrypt(nonce, bytes(bad), b"txn-header"); res["gcm_tamper"] = "accepted"
except Exception as e: res["gcm_tamper"] = "REJECTED (InvalidTag)"
# repeats count
res["ecb_dup"] = len(split(ecb)) - len(set(split(ecb))); res["cbc_dup"] = len(split(cbc)) - len(set(split(cbc)))

# ---- IoT benchmark ----
def timeit(f, n):
```

```
    t = time.perf_counter()
    for _ in range(n): f()
    return (time.perf_counter() - t) / n * 1000
msg = b"temperature=24.5C;door=locked"
rk = rsa.generate_private_key(65537, 2048); rpub = rk.public_key()
rsig = rk.sign(msg, padding.PKCS1v15(), hashes.SHA256())
ek = ec.generate_private_key(ec.SECP256R1()); epub = ek.public_key()
esig = ek.sign(msg, ec.ECDSA(hashes.SHA256()))
xa = x25519.X25519PrivateKey.generate(); xb = x25519.X25519PrivateKey.generate()
b = {}
b["RSA-2048 key generation"] = timeit(lambda: rsa.generate_private_key(65537, 2048), 5)
b["ECC P-256 key generation"] = timeit(lambda: ec.generate_private_key(ec.SECP256R1()), 200)
b["RSA-2048 sign (private key)"] = timeit(lambda: rk.sign(msg, padding.PKCS1v15(), hashes.SHA256()),
100)
b["ECDSA P-256 sign (private key)"] = timeit(lambda: ek.sign(msg, ec.ECDSA(hashes.SHA256())), 200)
b["RSA-2048 verify"] = timeit(lambda: rpub.verify(rsig, msg, padding.PKCS1v15(), hashes.SHA256()),
200)
b["ECDSA P-256 verify"] = timeit(lambda: epub.verify(esig, msg, ec.ECDSA(hashes.SHA256())), 200)
res["bench"] = b
res["sizes"] = {"rsa_pub": len(rpub.public_bytes(serialization.Encoding.DER,
serialization.PublicFormat.SubjectPublicKeyInfo)),
                "ecc_pub": len(epub.public_bytes(serialization.Encoding.DER,
serialization.PublicFormat.SubjectPublicKeyInfo)),
                "rsa_sig": len(rsig), "ecdsa_sig": len(esig), "x25519_pub": 32}
# ---- smart-home session: X25519 + HKDF + AES-GCM ----
sa = xa.exchange(xb.public_key()); sb = xb.exchange(xa.public_key())
assert sa == sb
k = HKDF(hashes.SHA256(), 16, None, b"smart-home session").derive(sa)
n2 = os.urandom(12)
c2 = AESGCM(k).encrypt(n2, msg, b"hub->lock")
p2 = AESGCM(k).decrypt(n2, c2, b"hub->lock")
res["session"] = {"shared_equal": sa == sb, "key_hex": k.hex(), "ct_len": len(c2), "pt_len": len(msg),
"roundtrip": p2.decode()}
json.dump(res, open("results.json", "w"), indent=1)
print(json.dumps(res, indent=1))
```


C:\WINDOWS\system32\CMD
Microsoft Windows [Version 10.0.26300.9711]
(c) Microsoft Corporation. All rights reserved.
C:\Users\RAKSHITH.R.R>pip install pycryptodome
Collecting pycryptodome
Downloading pycryptodome-4.2.0-cp39-abi3-win_amd64.whl.metadata (6.3 kB)
Downloading pycryptodome-4.2.0-cp39-abi3-win_amd64.whl (1.7 MB)
1.7/1.7 MB 1.1 MB/s eta 0:00:00
Installing collected packages: pycryptodome
Successfully installed pycryptodome-4.2.0
[notice] A new release of pip is available: 25.1.1 -> 26.2.2
[notice] To update, run: python.exe -m pip install --upgrade pip
C:\Users\RAKSHITH.R.R>pip install cryptography
Collecting cryptography
Downloading cryptography-50.0.2-cp311-abi3-win_amd64.whl.metadata (4.4 kB)
Collecting cffi>=2.0.0 (from cryptography)
Downloading cffi-2.1.1-cp312-cp312-win_amd64.whl.metadata (2.6 kB)
Requirement already satisfied: pycparser in c:\users\rakshith.r.r\appdata\local\programs\python\python312\lib\site-packages (from cffi>=2.0.0->cryptography)
(2.22)
Downloading cryptography-50.0.2-cp311-abi3-win_amd64.whl (3.8 MB)
3.8/3.8 MB 957.9 kB/s eta 0:00:00
Downloading cffi-2.1.1-cp312-cp312-win_amd64.whl (185 kB)
Installing collected packages: cffi, cryptography
Attempting uninstall: cffi
Found existing installation: cffi 1.17.1
Uninstalling cffi-1.17.1:
Successfully uninstalled cffi-1.17.1
Successfully installed cffi-2.1.1 cryptography-50.0.2
[notice] A new release of pip is available: 25.1.1 -> 26.2.2
[notice] To update, run: python.exe -m pip install --upgrade pip
C:\Users\RAKSHITH.R.R>
2:39 PM
10/29/2026


IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/RAKSHITH.R.R/Downloads/CO2 AT-2 CRYPTO.py
Squeezed text (64 lines).
>>>
except Exception as e: res["gcm_tamper"] = "REJECTED (InvalidTag)"
Ln: 6 Col: 0
ENG
IN
2:39 PM
10/29/2026



Environment: Python 3 with the cryptography library (version 46). Key and nonce values in the program are fixed for repeatability; random values are used only where the program says so.